BookMyShow

Movie Ticket Booking Application — Project Report

Developed using Microsoft Power Apps (Canvas App)
Backend: Microsoft SharePoint Lists

Project Overview

BookMyShow is a movie ticket booking application built on Microsoft Power Apps (Canvas App), designed to replicate the core experience of a real-world ticket booking platform. The app covers the complete user journey — from registration and login, through browsing movies, booking seats, ordering food and drinks, and redeeming offers.

The application front-end has 14 screens built in Power Apps, and the data behind it is stored and managed using Microsoft SharePoint Lists, which act as the backend database. Each screen is built using standard Canvas App controls such as Text Input, Button, Label, Gallery, Icon, and Image controls, connected to SharePoint through Power Apps formulas like Patch, SubmitForm, Filter, and Navigate.

This report documents each Power Apps screen along with the corresponding SharePoint lists that store and manage the data, and explains the purpose of the key buttons, labels, and text input fields on every screen.

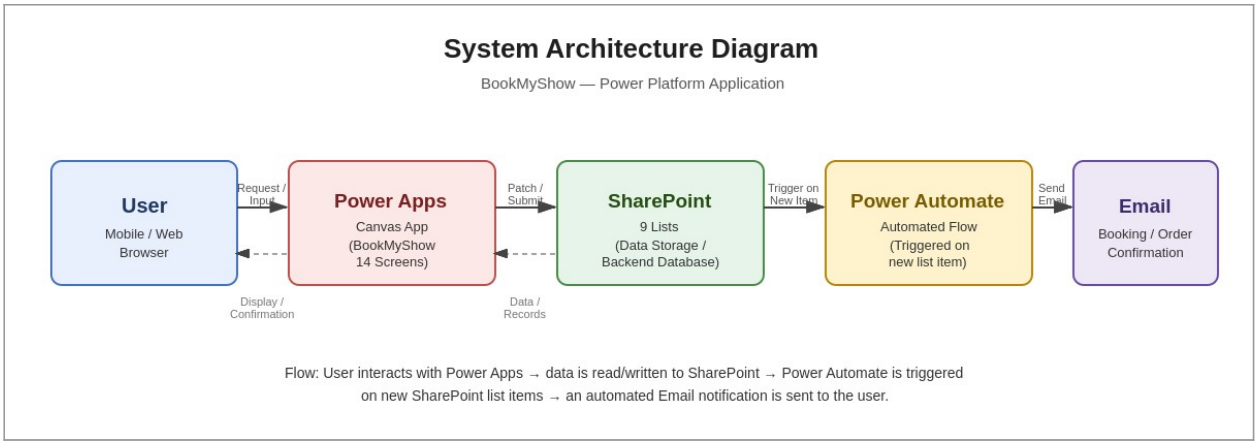
Report Structure

Part A — Power Apps Screens (14 screens): the front-end user interface, app flow, and control-level breakdown.

Part B — SharePoint Backend Lists (9 lists): the data storage layer supporting the app.

System Architecture Diagram

How the major components of the solution connect and pass data end to end.



The BookMyShow solution is built entirely on the Microsoft Power Platform, using four connected layers. The User interacts with the app through a phone or web browser, which is the entry point for every action in the system.

Power Apps (the Canvas App) is the front-end layer containing all 14 screens. It captures user input through Text Input and Button controls, and displays data pulled from the backend using Gallery and Label controls.

SharePoint is the backend data layer. All 9 lists — Login Details, Movies, Booking Ticket, the food and drink lists, and the offers lists — live here and are read from and written to using Power Fx formulas such as Patch, Filter, and LookUp.

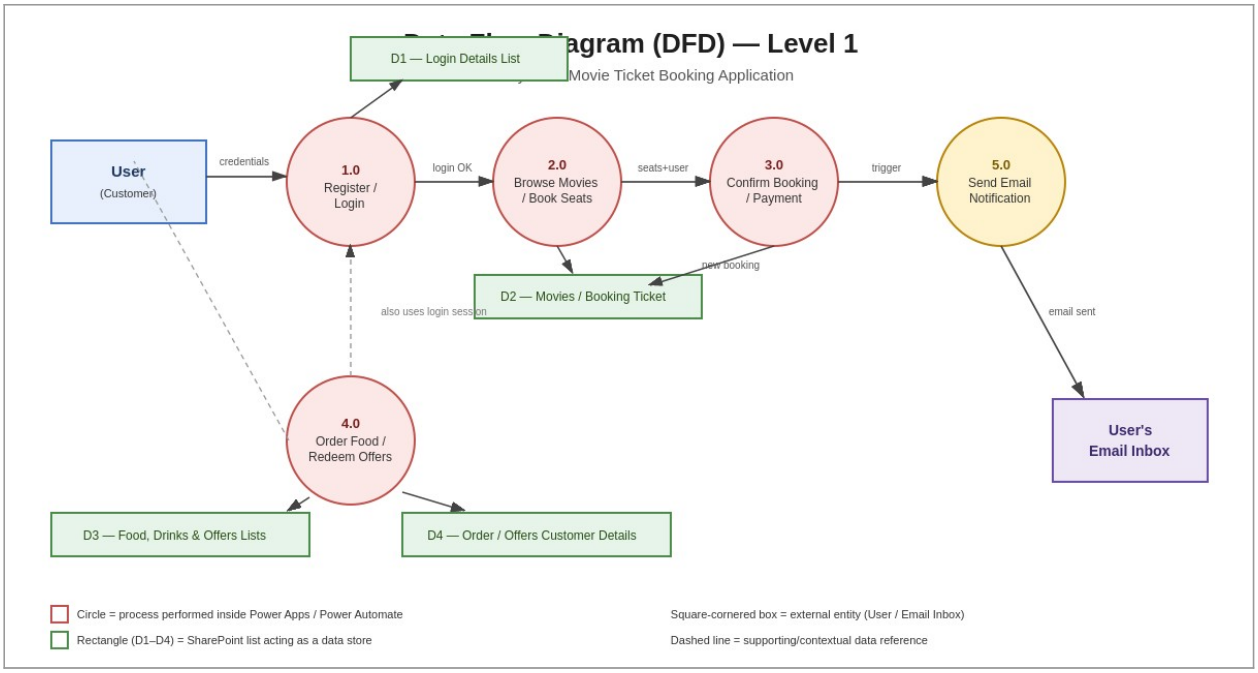
Power Automate sits behind SharePoint and is triggered automatically whenever a new item is created in a key list, such as a new row in the Booking Ticket list or the Food & Drinks Order list.

The final step of the flow is Email: the Power Automate flow sends an automated confirmation email to the user once their booking or order has been successfully recorded, closing the loop from user action to user notification.

This layered architecture means each part of the system has a single clear responsibility — Power Apps handles the interface, SharePoint handles storage, and Power Automate handles automation — which is a good structure to describe in an interview when asked how the different Power Platform tools fit together.

Data Flow Diagram (DFD)

A Level-1 DFD showing how data moves between the user, the app's processes, and the SharePoint data stores.



This Data Flow Diagram breaks the app down into five core processes: Register/Login, Browse Movies & Book Seats, Confirm Booking & Payment, Order Food & Redeem Offers, and Send Email Notification.

The User is shown as an external entity on the left, providing inputs such as login credentials, seat selections, and food/offer choices, and receiving outputs such as confirmations and the final email.

Process 1.0 (Register/Login) reads from and writes to the Login Details list (D1), which is why it is the very first process any user must pass through before other processes become available.

Process 2.0 (Browse Movies/Book Seats) reads movie data and writes new booking records into D2 (Movies / Booking Ticket), while Process 3.0 (Confirm Booking/Payment) finalises that booking.

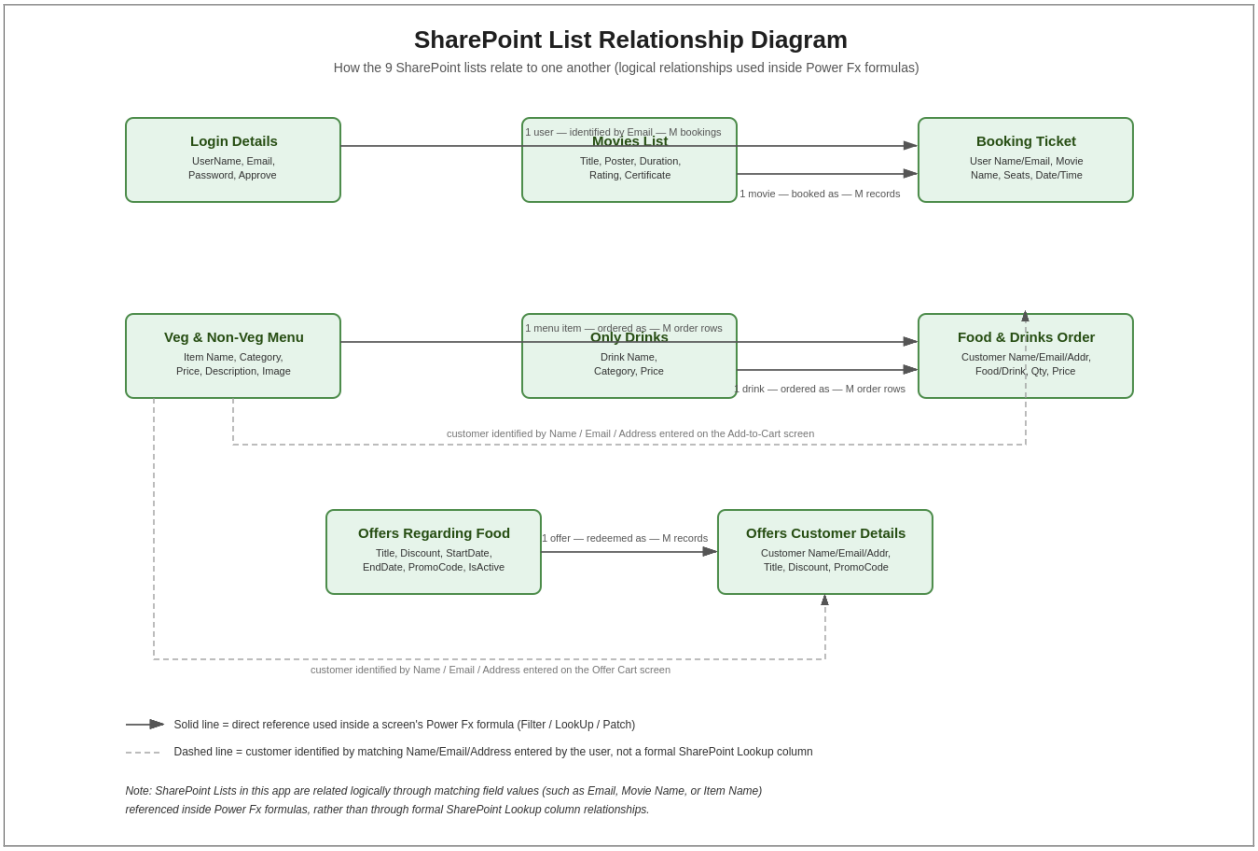
Process 4.0 (Order Food/Redeem Offers) interacts with D3 (Food, Drinks & Offers Lists) to display the menu and offers, and writes completed orders and redemptions into D4 (Order/Offers Customer Details).

Process 5.0 (Send Email Notification) is triggered once a new record lands in the booking or order data stores, and its only output is the email delivered to the user's inbox, shown as a separate external entity on the right.

This diagram is a good one to walk through in an interview because it shows the same system as the architecture diagram, but from a data-movement perspective rather than a component perspective.

SharePoint List Relationship Diagram

How the 9 SharePoint lists relate to one another through shared field values referenced in Power Fx formulas.



The 9 SharePoint lists are related logically rather than through formal SharePoint Lookup columns — the app matches records across lists using shared values such as Email, Movie Name, or Item Name inside Power Fx formulas.

The Login Details list sits at the centre of the authentication side of the app: it is referenced from the Booking Ticket list (to identify which user made a booking) and from the Offers Customer Details list (to identify who redeemed an offer).

The Movies list has a one-to-many relationship with the Booking Ticket list, since a single movie can appear in many different booking records across different users, dates, and showtimes.

Similarly, the Veg & Non-Veg Menu list and the Only Drinks list each have a one-to-many relationship with the Food & Drinks Order list, since one menu item or drink can be ordered many times by different customers.

The Offers Regarding Food list has a one-to-many relationship with the Offers Customer Details list: one active offer can be redeemed by many different customers, each creating its own row in the customer details list.

Because there are no enforced relationships, data integrity depends entirely on the accuracy of the Power Fx formulas — a limitation worth mentioning in an interview alongside how it could be improved with Dataverse relationships in production.

Power Fx Formulas Used in This App

Power Fx is the formula language used throughout Power Apps to control navigation, read and write data, and manage temporary in-memory data such as the shopping cart. The table below lists the key functions used across the BookMyShow app's screens, with a short explanation of how each one was applied.

Patch

Purpose: creates a new record in a SharePoint list, or updates an existing one, without needing a full form.

Used on: New Register screen (creating a new Login Details record), Reset Password screen (updating the Password field), Confirm Booking screen (creating a Booking Ticket record), Add to Cart screen (creating Food & Drinks Order records), and Offer Cart screen (creating an Offers Customer Details record).

```
Example: Patch('Login Details', Defaults('Login Details'), {UserName:
txtName.Text, Customer_Email: txtEmail.Text, Customer_Password:
txtPassword.Text})
```

LookUp

Purpose: retrieves a single matching record from a list, most often used to check credentials during login.

Used on: Login Page, to find the one Login Details record whose Email and Password match what the user typed in, before allowing navigation to the Home Screen.

```
Example: LookUp('Login Details', Customer_Email = txtEmail.Text &&
Customer_Password = txtPassword.Text)
```

Filter

Purpose: returns a filtered set of records from a list, used to control what appears inside a Gallery control.

Used on: the Offers screen, so only offers where IsActive is true are shown to the user; also usable on the Movies screen to filter by language or certificate if needed.

```
Example: Filter('Offers Regarding Food', IsActive = true)
```

Navigate

Purpose: moves the user from one screen to another, optionally with a screen transition animation.

Used on: almost every screen in the app — for example, the LOGIN and NEW REGISTER buttons on the landing screen, the Book Ticket button on the Book Movie screen, and the PAY button on the Seat Booking screen.

```
Example: Navigate('Home Screen', ScreenTransition.Fade)
```

Collect / ClearCollect

Purpose: builds up an in-memory collection of records during the session, such as items added to the food and drinks cart, before finally saving them to SharePoint.

Used on: the Food & Drink screen, to add each selected item into a temporary cart collection; ClearCollect is used to reset the cart back to empty once the order has been submitted from the Add to Cart screen.

```
Example: Collect(colCart, {Item: ThisItem.ItemName, Qty: 1, Price:
ThisItem.Price})   /   ClearCollect(colCart, [])
```

Together, these six functions cover the core Power Fx skills expected in most Power Platform developer interviews: reading a single record (LookUp), reading multiple filtered records (Filter), writing or updating a record (Patch), moving between screens (Navigate), and managing temporary local data before committing it (Collect/ClearCollect).

Part A: Power Apps Screens

The following 14 screens make up the Canvas App front-end, covering registration, login, browsing, booking, payment, food ordering, and offers. Each screen description also lists the main controls used (buttons, labels, text inputs, and galleries) and what each one does.

Screen 1 of 14

1. Who Are You? (Landing / Role Selection Screen)

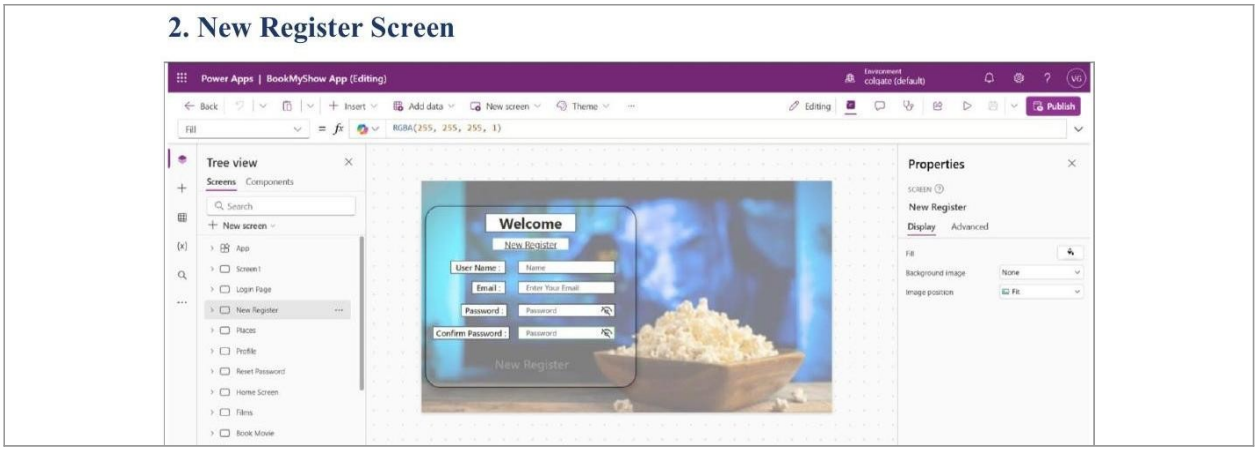
This is the entry screen of the app, and it is the first thing a user sees when the application is opened. The design uses a full-screen background image of a popcorn bowl to immediately set a cinematic, movie-going tone. A large label at the top displays the heading 'WHO ARE YOU?', prompting the user to identify themselves as either a new or returning customer. Two prominent buttons are placed side by side in the centre of the screen: LOGIN and NEW REGISTER. The LOGIN button uses a Navigate function in its OnSelect property to move the user to the Login Page screen. The NEW REGISTER button similarly uses a Navigate function to take first-time users to the New Register screen. This screen effectively acts as a router or decision point, so the app does not need to guess whether a user already has an account. No text input or data entry happens on this screen; it is purely a navigation screen. Keeping this screen simple with only two choices reduces confusion for first-time users and speeds up onboarding. From a Power Apps development perspective, this screen typically contains only Label, Button, and Image controls, and no data connection is required here since nothing is being read or written yet.

Key controls used:

Label123 (heading text 'WHO ARE YOU?'), NEW REGISTER BUTTON and LOGIN BUTTON (navigation buttons using Navigate()), Image56 (background image).

Screen 2 of 14

2. New Register Screen



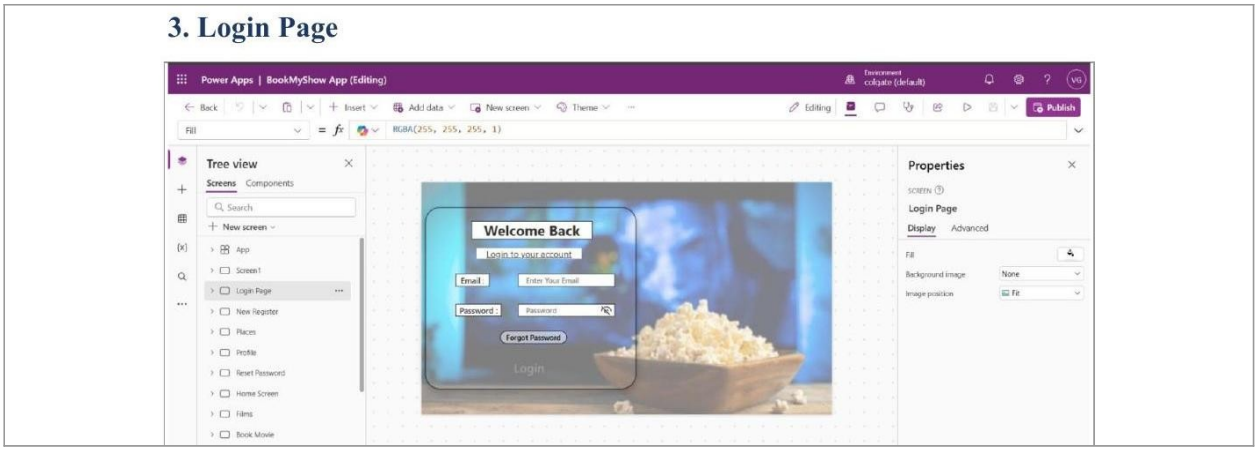
The New Register screen allows a first-time user to create a new account inside the app. A label at the top titled 'Welcome' and a sub-label 'New Register' clearly identify the purpose of the screen to the user. Four text input boxes are provided: User Name, Email, Password, and Confirm Password, each preceded by a label describing the field. The Password and Confirm Password fields use the Password mode property of the Text Input control so the characters typed are masked for privacy. An eye icon next to the password fields allows the user to toggle visibility of the password if needed. When the user submits the form, a validation check compares the Password and Confirm Password fields to ensure they match exactly. If validation passes, the entered details are written into the Login Details SharePoint list using a Patch or SubmitForm function. If the passwords do not match, an error message or notification is typically shown to the user instead of saving the record. Once the account is created successfully, the user can go back to the Login Page and sign in using the same credentials. This screen demonstrates basic form validation logic, which is one of the most commonly asked topics in a Power Apps interview. The screen also reuses the same cinematic background as the landing screen, keeping the visual theme consistent across the registration flow.

Key controls used:

Text Input controls for User Name, Email, Password, and Confirm Password; a Register/Submit Button that runs Patch() to the Login Details list; Label controls for the 'Welcome' title and field captions.

Screen 3 of 14

3. Login Page



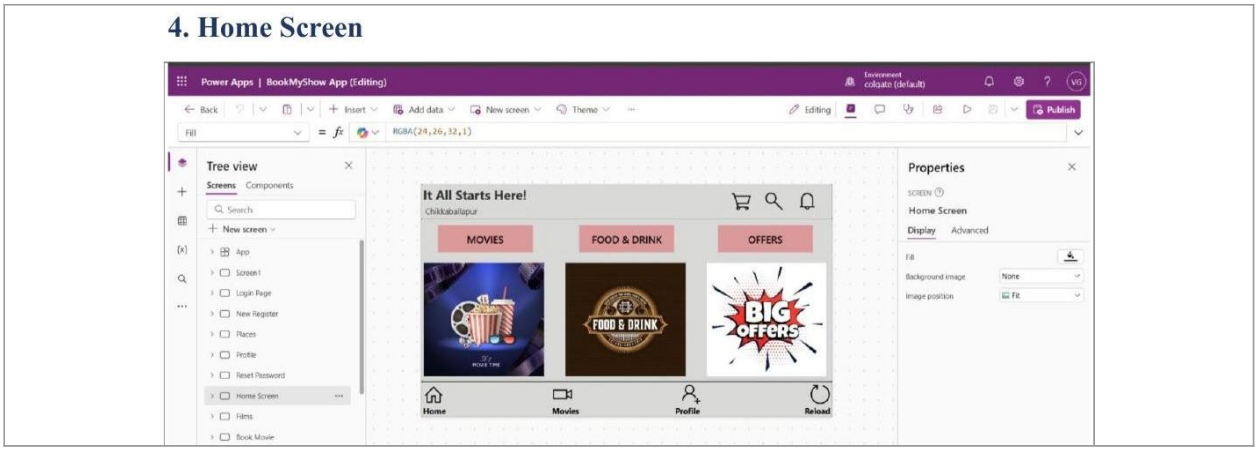
The Login Page is used by existing, already-registered users to access their account. A 'Welcome Back' label at the top and a sub-label 'Login to your account' clearly indicate the purpose of the screen. Two text input fields are provided: Email and Password, allowing the user to enter their registered credentials. The Password field is masked using the Password mode, with an eye icon to reveal or hide the typed password. The main Login button runs a formula that checks the entered Email and Password against records stored in the Login Details SharePoint list using a Filter or LookUp function. If the credentials match an existing record, the Navigate function moves the user to the Home Screen and the session is treated as logged in. If the credentials do not match, the app is expected to show an error notification instead of navigating forward. A 'Forgot Password' link/button is also placed on this screen, which uses Navigate to redirect the user to the Reset Password screen. This screen is a critical security checkpoint in the app, since it is the gate that decides whether a user can reach any of the protected screens. In interviews, this screen is a good example to explain how Power Apps can perform simple authentication without a dedicated backend using only SharePoint list lookups. The same background image is reused here again to maintain visual consistency with the Landing and Register screens.

Key controls used:

Text Input controls for Email and Password; Login Button using LookUp()/Filter() against the Login Details list plus Navigate(); Forgot Password link/button navigating to Reset Password screen.

Screen 4 of 14

4. Home Screen



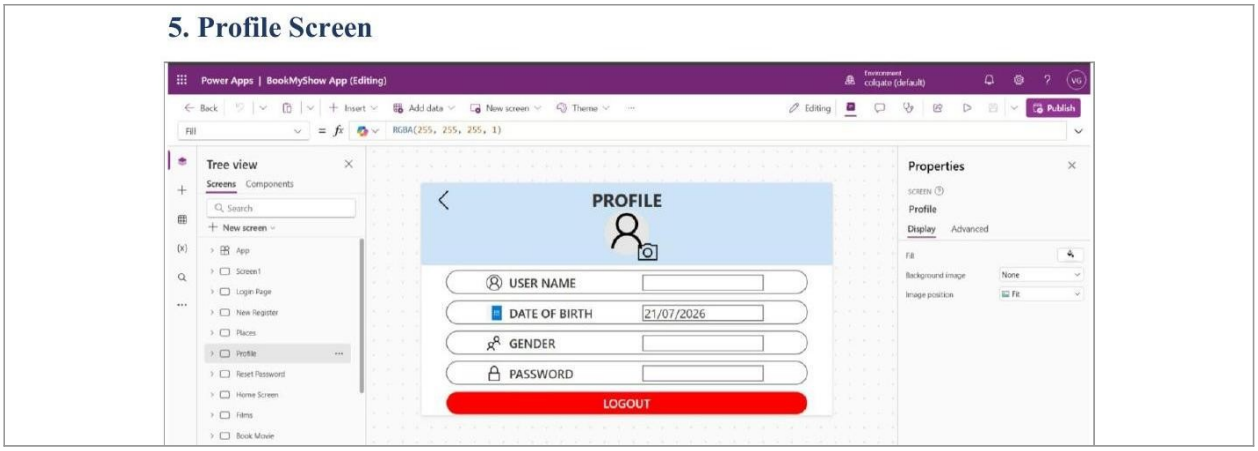
The Home Screen acts as the central hub of the application once a user has successfully logged in. A label at the top displays the tagline 'It All Starts Here!' along with the user's current location, shown here as Chikkaballapur. Three large tile-style buttons are placed in the middle of the screen for Movies, Food & Drink, and Offers, each represented with its own icon or image. Selecting the Movies tile navigates the user to the Films screen where all available movies are listed. Selecting the Food & Drink tile navigates the user to the food ordering flow, and selecting Offers takes the user to the promotional deals screen. A cart icon, search icon, and notification icon are placed at the top-right corner, giving quick access to the shopping cart, search functionality, and alerts. A bottom navigation bar with four icons — Home, Movies, Profile, and Reload — is fixed at the bottom of the screen for quick access from anywhere in the app. The Reload icon is typically wired to the Power Apps Refresh() function to reload data from SharePoint in case anything has changed. This screen uses mostly Icon, Button, and Label controls, with minimal text input since its primary role is navigation rather than data entry. Structuring the app around a central Home Screen like this is a common design pattern that keeps navigation intuitive and reduces the number of clicks needed to reach any major feature. This is usually one of the best screens to walk an interviewer through, since it shows how the whole app's navigation architecture is organised.

Key controls used:

MOVIES, FOOD & DRINK, and OFFERS tile buttons using Navigate(); bottom nav icons for Home, Movies, Profile, Reload; cart/search/notification icons; Label for location and tagline text.

Screen 5 of 14

5. Profile Screen



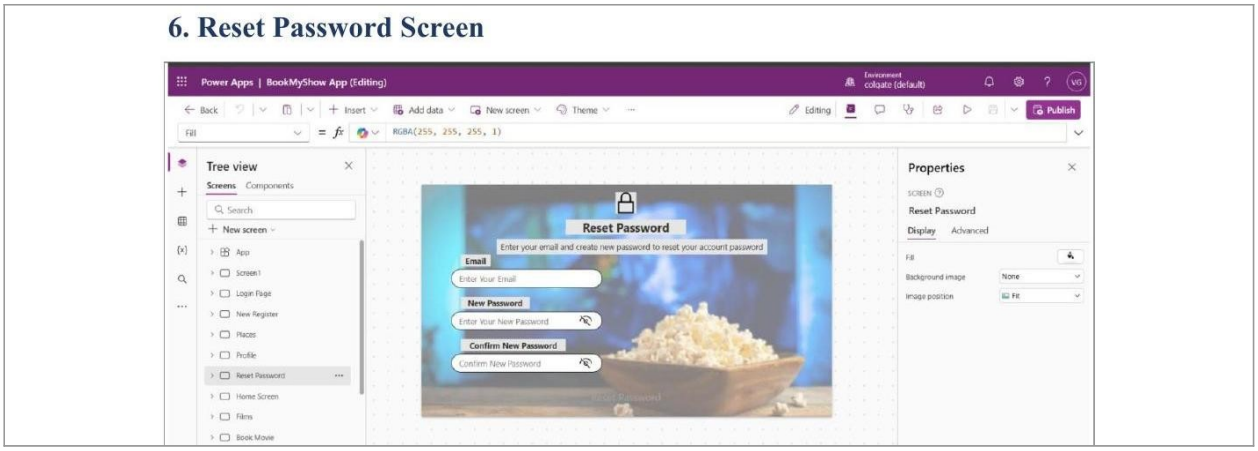
The Profile screen allows a logged-in user to view and manage their personal account information. A back arrow icon at the top-left lets the user return to the previous screen without logging out. A profile picture placeholder with a small camera icon allows the user to update their profile photo. Four labelled text input fields are shown: User Name, Date of Birth, Gender, and Password, each pre-filled with the user's existing data pulled from the Login Details SharePoint list. The Date of Birth field uses a date-formatted text input, shown here as 21/07/2026, to keep the format consistent. These fields are typically editable, so the user can update their details and the app can Patch the changes back into the SharePoint list. A large red LOGOUT button is placed at the bottom of the screen, styled in a warning colour to visually separate it from the informational fields above. The LOGOUT button's OnSelect property clears any session variables that were storing the logged-in user's details and then navigates back to the Landing screen. This screen demonstrates how Power Apps can be used to both display and edit existing records from a data source, not just create new ones. It is a useful screen to discuss in an interview when explaining variable management, since the app needs to remember which user is logged in throughout the session using a global variable. Keeping the Profile screen separate from the Home Screen also keeps the app's main navigation uncluttered.

Key controls used:

Text Input controls for User Name, Date of Birth, Gender, Password (pre-populated from data source); LOGOUT Button clearing session variables and navigating to Landing screen; back-arrow icon for Navigate(Back).

Screen 6 of 14

6. Reset Password Screen



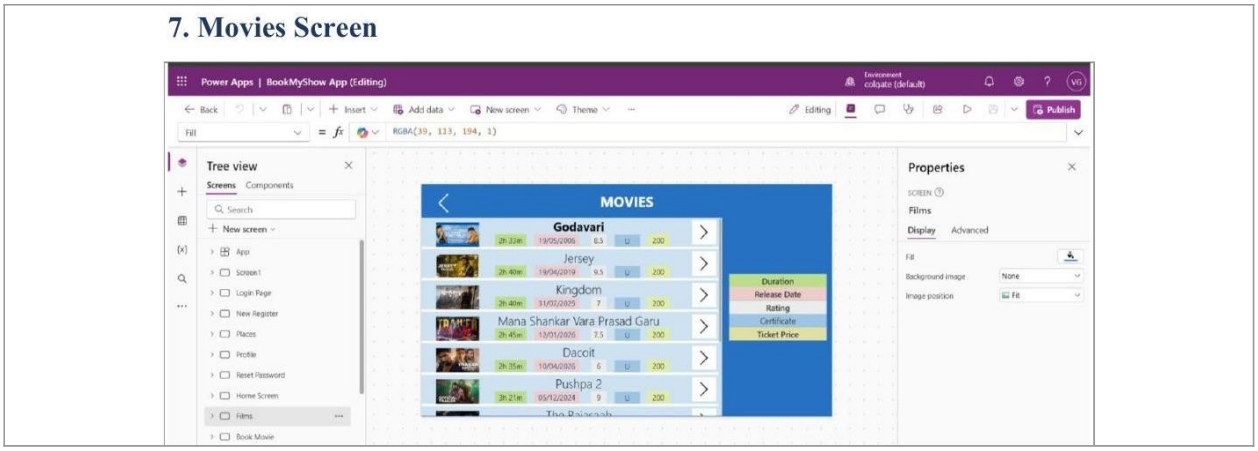
This screen enables a user who has forgotten their password to regain access to their account. A lock icon and the heading label 'Reset Password' at the top clearly communicate the purpose of the screen. A sub-label explains the process in plain language: 'Enter your email and create new password to reset your account password.' The user first enters their registered Email address in a text input field so the app can identify which account record to update. Two more text input fields follow: New Password and Confirm New Password, both using masked password mode with an eye icon to toggle visibility. When the Reset Password button is selected, the app validates that the New Password and Confirm New Password fields match. If they match, the app runs a Patch function that updates the Password field of the matching record in the Login Details SharePoint list, identified using the entered Email. If the Email entered does not match any record, the app is expected to show an error rather than attempting to update anything. Once the password is successfully changed, the user is usually navigated back to the Login Page to sign in with the new password. This is another screen that is excellent for discussing form validation and conditional Patch statements in a Power Apps interview. The screen reuses the same background theme as the Login and Register screens, keeping the authentication flow visually unified.

Key controls used:

Text Input controls for Email, New Password, Confirm New Password; Reset Password Button running Patch() on the matched record; lock icon and instructional Label.

Screen 7 of 14

7. Movies Screen



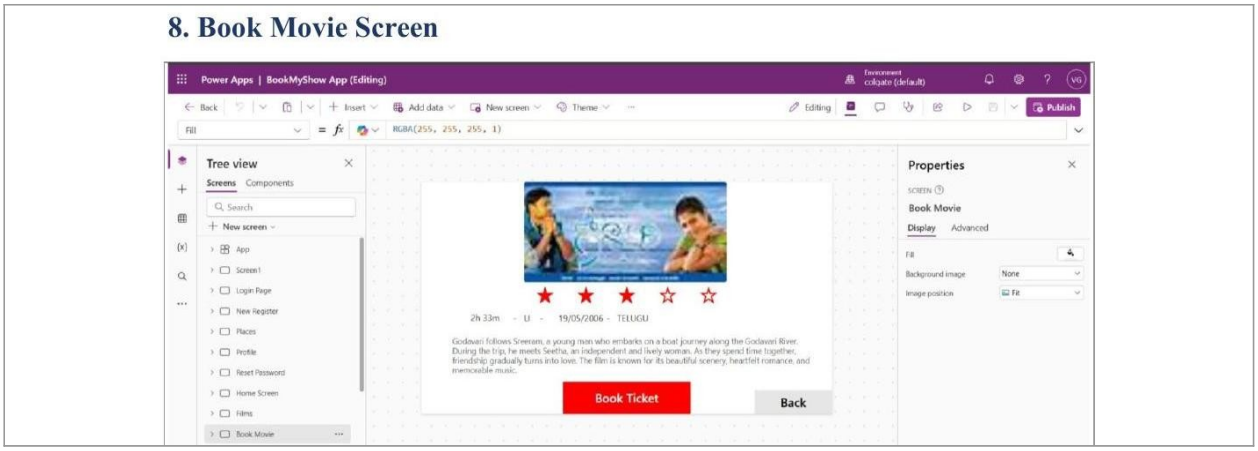
The Movies screen, internally named 'Films', lists all currently available movies that users can book tickets for. A back arrow at the top-left allows the user to return to the Home Screen, and a centred label displays the heading 'MOVIES'. The main part of the screen is a vertical Gallery control that repeats a template for each movie record pulled from the Movies SharePoint list. Each gallery item displays the movie poster image, title, duration, release date, a rating value, an age certificate, and the ticket price. Movies visible in this list include titles such as Godavari, Jersey, Kingdom, Dacoit, and Pushpa 2, each shown as a separate gallery row. A small coloured legend box on the right side of the screen labels each column (Duration, Release Date, Rating, Certificate, Ticket Price) for clarity. A chevron/arrow icon on the right of each gallery item acts as a button that navigates the user to the Book Movie screen for that specific film. The OnSelect property of each gallery item typically sets a context variable holding the selected movie record, which the next screen then reads. Because the Gallery is bound directly to the SharePoint Movies list, any new movie added to the list automatically appears here without changing the app's code. This screen is a strong example to highlight in an interview when explaining how Power Apps Gallery controls connect to and display SharePoint data dynamically. The scrollable gallery design also means the list can grow indefinitely without redesigning the screen.

Key controls used:

Gallery control bound to the Movies SharePoint list; chevron/arrow icon buttons per item using Navigate() with a selected-record context variable; Labels for Duration, Release Date, Rating, Certificate, Ticket Price.

Screen 8 of 14

8. Book Movie Screen



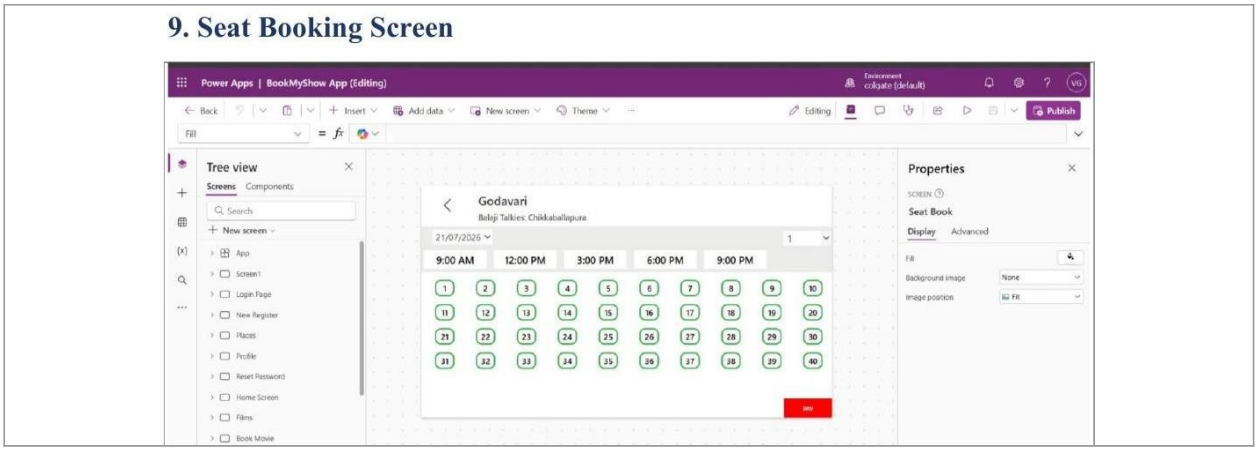
The Book Movie screen opens after a user selects a specific film from the Movies gallery. A large image control at the top displays the poster of the selected movie, pulled dynamically from the record chosen on the previous screen. A star rating display, built using five Icon controls, visually shows the movie's rating out of five stars, with filled and outline stars representing the score. Labels beneath the poster display key details in a single line: duration, certificate, release date, and language, for example '2h 33m - U - 19/05/2006 - TELUGU'. A longer text label below that shows the movie's synopsis or short story description, giving the user enough context to decide whether to watch it. A prominent red 'Book Ticket' button is placed at the bottom of the screen, and its OnSelect property navigates the user forward to the Seat Booking screen. A secondary 'Back' button next to it allows the user to return to the Movies screen without proceeding to book a ticket. All of the text and image values on this screen are dynamic, meaning they change automatically depending on which movie the user tapped on the previous screen. This dynamic behaviour is achieved by referencing a single selected-record variable in every Label and Image control's Text or Image property. This screen is useful to describe in an interview when explaining how a single screen template can be reused for many different data records instead of creating a separate screen per movie. It also shows a clean example of passing context between screens in Power Apps.

Key controls used:

Image control bound to selected movie's poster; five-star rating built from Icon controls; Labels for duration/certificate/release date/language and synopsis; Book Ticket Button (Navigate to Seat Book) and Back Button (Navigate Back).

Screen 9 of 14

9. Seat Booking Screen



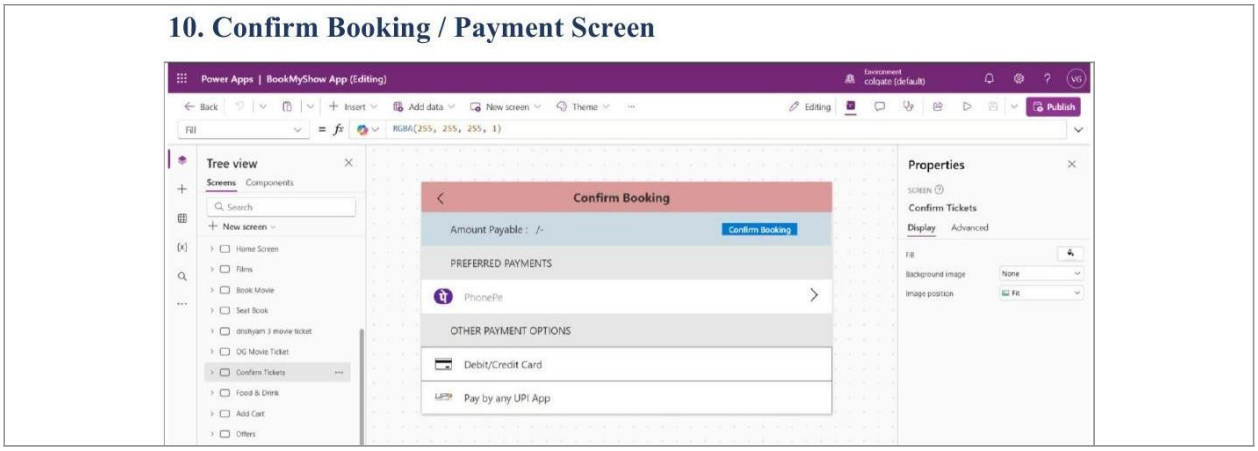
The Seat Booking screen, named 'Seat Book' in the app, is where the user actually selects their show timing and seats. At the top, labels display the movie name (Godavari in this example) and the theatre name and location, 'Balaji Talkies: Chikkaballapura'. A date picker control shows the selected show date, here 21/07/2026, allowing the user to change the date if multiple dates are available. A row of five time-slot buttons — 9:00 AM, 12:00 PM, 3:00 PM, 6:00 PM, and 9:00 PM — lets the user pick their preferred show timing, with the selected button usually highlighted. A dropdown control next to the date picker lets the user choose the number of tickets they want to book. The main body of the screen is a grid of 40 numbered seat buttons, arranged in rows, representing every seat in the theatre for that show. Each seat button toggles between an available and selected state when tapped, usually changing colour (for example, green outline for available, filled colour for selected). The number of seats the user can select is typically validated against the ticket-count dropdown chosen earlier, so the user cannot select more seats than tickets requested. A red 'PAY' button at the bottom becomes active once the required number of seats is selected, and its OnSelect property navigates to the Confirm Booking / Payment screen while carrying forward the selected seat numbers. This screen is one of the most interactive parts of the whole app and is an excellent talking point in an interview for demonstrating dynamic UI state changes based on user selection. Behind the scenes, the selected seats and showtime are usually stored in collection or context variables so they can be written to the Booking Ticket SharePoint list once payment is confirmed.

Key controls used:

Date picker and time-slot Buttons (9 AM–9 PM); ticket-count Dropdown; 40 numbered seat toggle Buttons; PAY Button navigating to Confirm Booking screen while passing selected seat data forward.

Screen 10 of 14

10. Confirm Booking / Payment Screen



This screen, named 'Confirm Tickets' internally, finalises the ticket booking process after seats have been selected. A back arrow at the top-left and a centred label 'Confirm Booking' identify the screen's purpose to the user. A label shows 'Amount Payable' along with the calculated total price, which is derived from the number of seats selected multiplied by the ticket price. A 'Confirm Booking' button is placed at the top-right of this summary bar, which commits the booking once a payment method has been chosen. Below that, a 'Preferred Payments' section lists PhonePe as a quick option, shown with its icon and a forward arrow to expand further payment details. An 'Other Payment Options' section lists two more choices: Debit/Credit Card and Pay by any UPI App, each shown as a selectable row with an icon. Selecting a payment method typically sets a variable recording which payment option the user chose, though real payment processing is outside the scope of a Power Apps prototype. Once 'Confirm Booking' is selected, the app writes a new record into the Booking Ticket SharePoint list containing the user's details, chosen movie, showtime, and seat numbers. After the record is successfully saved, the user is usually navigated to a confirmation message or back to the Home Screen. This screen is a good example to discuss in an interview for explaining how a Power Apps prototype can simulate a real payment gateway UI without needing an actual payment integration. It also demonstrates writing a final transactional record to SharePoint using a Patch or SubmitForm function after several screens of data collection.

Key controls used:

Label showing calculated Amount Payable; Confirm Booking Button running Patch() into the Booking Ticket list; selectable payment-option rows for PhonePe, Debit/Credit Card, and UPI, each with an Icon and forward-arrow.

Screen 11 of 14

11. Food & Drink Screen

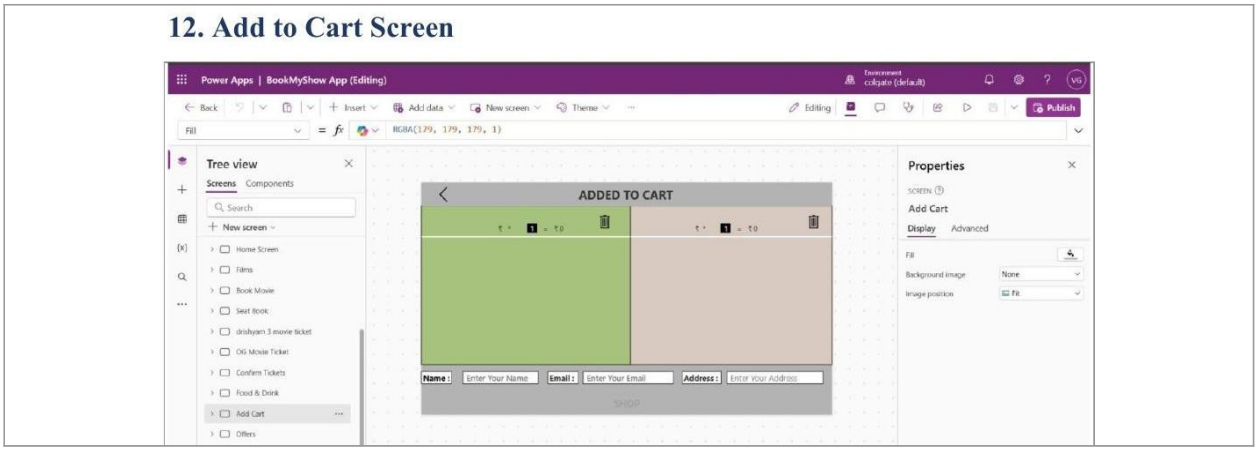
The Food & Drink screen lets users browse and choose snacks and beverages to add on to their movie booking. A back arrow and a centred label 'Food & Drink' sit at the top of the screen, consistent with the navigation pattern used across the app. The screen is split into two vertical Gallery controls side by side: one listing food items and the other listing drinks. The food gallery, bound to the Veg & Non-Veg Menu SharePoint list, shows items such as Veg Burger, Margherita Pizza, Paneer Roll, Grilled Cheese Sandwich, Veg Momos, and Chicken Burger, each with its category and price. The drinks gallery, bound to the Only Drinks SharePoint list, shows items such as Pepsi, Coca-Cola, Sprite, Orange Juice, Mango Juice, and Pineapple Juice, each labelled with its category and price. Each row in both galleries has a chevron/arrow icon button that lets the user open the item and choose a quantity, which then gets added to a running cart collection. A 'Next' button spans the width of the screen at the bottom, and its OnSelect moves the user forward to the Add to Cart screen once they are done selecting items. Since both galleries pull live from SharePoint, adding or removing a menu item in the list automatically updates what customers see without touching the app itself. This screen is a strong example to bring up in an interview when explaining how two independent Gallery controls can be placed on the same screen, each bound to a different data source. It also shows good UI practice by grouping food and drinks separately rather than mixing them into a single long list. The colour-coded rows (green for food, orange for drinks) help users visually separate the two categories at a glance.

Key controls used:

Two Gallery controls bound to the Veg & Non-Veg Menu list and the Only Drinks list; chevron/arrow icon Buttons per item for adding to a cart collection; Next Button navigating to Add Cart screen.

Screen 12 of 14

12. Add to Cart Screen

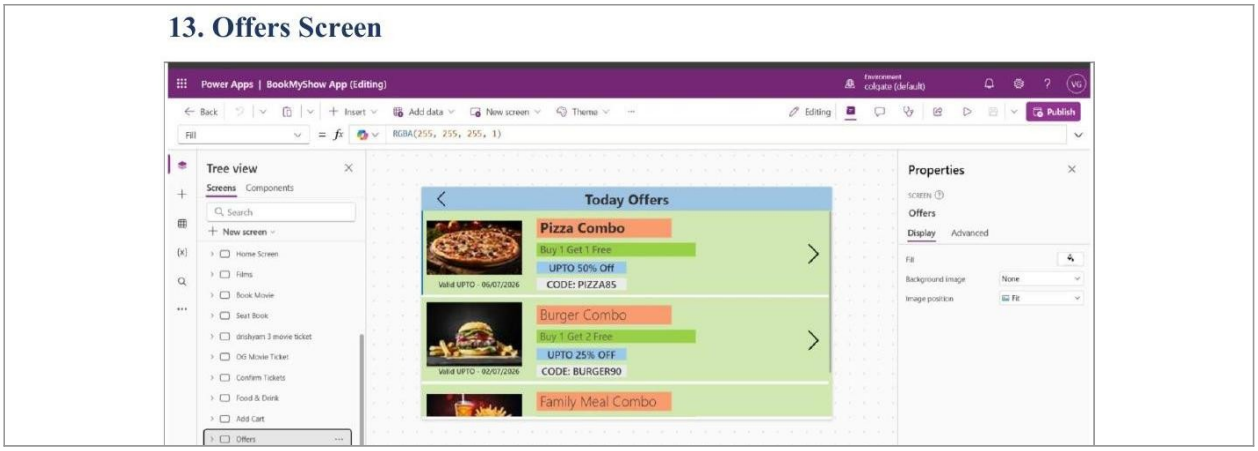


The Add to Cart screen, named 'Add Cart' in the app, displays everything the user has selected on the Food & Drink screen before checkout. A back arrow and the centred label 'ADDED TO CART' identify the screen's purpose at the top. The cart items are displayed using two panel/gallery sections side by side, each showing an item's quantity control and its running subtotal. A small trash/delete icon button is placed above each item panel, letting the user remove that item from the cart entirely. A quantity stepper, shown as a numeric value with plus and minus controls, lets the user increase or decrease how many of each item they want before finalising the order. Below the cart items, three text input fields are provided: Name, Email, and Address, which capture the customer's delivery and contact details. These text inputs are required fields, since the order confirmation and any delivery communication depend on accurate contact information. A 'SHOP' button spans the bottom of the screen, and selecting it triggers a Patch or Collect function that writes each cart item, along with the customer's Name, Email, and Address, into the Food & Drinks Order SharePoint list. Once the order is saved, the cart collection is typically cleared and the user is navigated back to the Home Screen or shown a confirmation message. This screen is useful in an interview for discussing how Power Apps handles working with local collections (the cart) before finally committing data to a permanent SharePoint list. It also demonstrates combining both read (displaying cart items) and write (submitting the order) operations on a single screen.

Key controls used:

Quantity stepper controls (plus/minus Buttons) per cart item; trash icon Buttons to remove items; Text Input controls for Name, Email, Address; SHOP Button writing all cart records to the Food & Drinks Order list.

13. Offers Screen



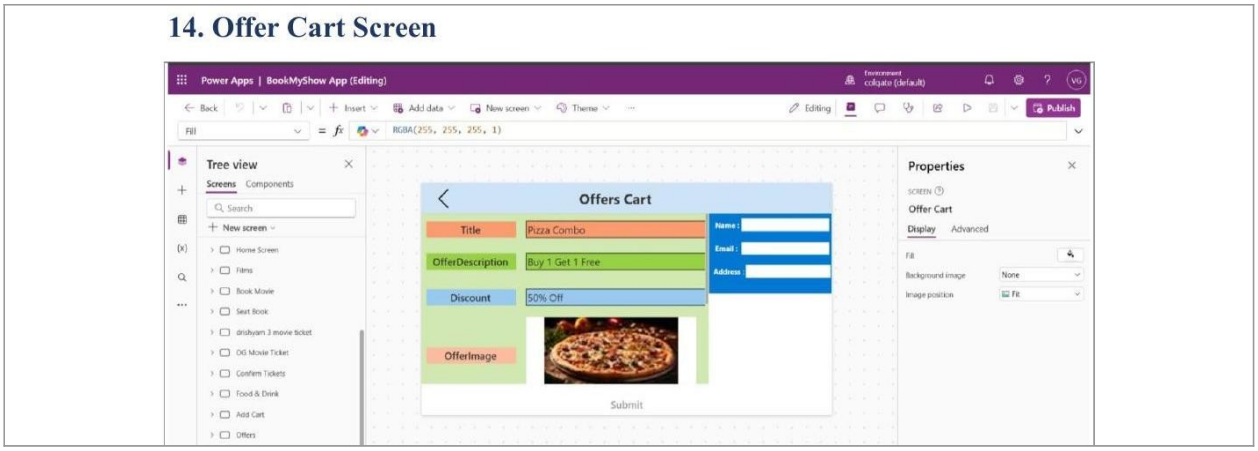
The Offers screen, simply named 'Offers' in the app, displays ongoing promotional deals and combo offers available to users. A back arrow and the centred label 'Today Offers' sit at the top, following the same layout pattern used on other list-style screens. The main content is a vertical Gallery control bound to the Offers Regarding Food SharePoint list, showing one row per active offer. Each gallery row displays an offer image, a bold title such as 'Pizza Combo' or 'Burger Combo', a short offer description, the discount percentage, a validity date, and a promo code. For example, the Pizza Combo offer shows 'Buy 1 Get 1 Free', 'UPTO 50% Off', valid until 06/07/2026, with the code PIZZA85. A chevron/arrow icon on the right side of each row acts as a button that navigates the user to the Offer Cart screen for that specific offer. The gallery only displays offers where the IsActive field in the underlying SharePoint list is set to true, so expired or disabled offers are automatically hidden without deleting them. This filtering is usually done with a Filter function inside the gallery's Items property, referencing the IsActive column and possibly the EndDate column too. This screen is a good example to raise in an interview for explaining how conditional filtering on a SharePoint list column can control what end users see, without needing to change the app. It also shows consistent reuse of the same gallery-plus-navigation pattern already used on the Movies and Food & Drink screens, which is good practice for maintainability. Keeping offers in their own dedicated list, separate from the food menu, also keeps the promotional content easy for an admin to manage independently.

Key controls used:

Gallery control bound to the Offers Regarding Food list, filtered by the IsActive column; chevron/arrow icon Buttons per offer navigating to Offer Cart; Labels for Title, Offer Description, Discount, and Promo Code.

Screen 14 of 14

14. Offer Cart Screen



The Offer Cart screen shows the full details of a single offer that the user selected from the Offers screen, and lets them redeem it. A back arrow and the centred label 'Offers Cart' appear at the top, keeping the same navigation style as the rest of the app. On the left side, four labelled rows display the selected offer's Title, Offer Description, Discount percentage, and Offer Image, each pulled from the record chosen on the previous screen. In the example shown, these fields display 'Pizza Combo', 'Buy 1 Get 1 Free', '50% Off', and the corresponding pizza image. On the right side, three text input fields — Name, Email, and Address — let the user enter their details in order to redeem the offer. These fields are required since the redeemed offer record needs to be tied to a specific customer for tracking and validation. A 'Submit' button spans the bottom of the screen, and its OnSelect property runs a Patch function that writes a new record into the Offers Customer Details SharePoint list. The new record includes the customer's Name, Email, Address, along with the offer Title, Description, Discount, and Image reference, effectively logging who redeemed which offer. Once submitted, the user is typically shown a confirmation notification or navigated back to the Home Screen. This screen closes out the offers flow, mirroring the same read-then-write pattern already seen on the Add to Cart screen, which is a good pattern to point out in an interview for consistency in app design. It is also a clean example of passing a selected record from one screen to another and then combining it with freshly entered user input before saving.

Key controls used:

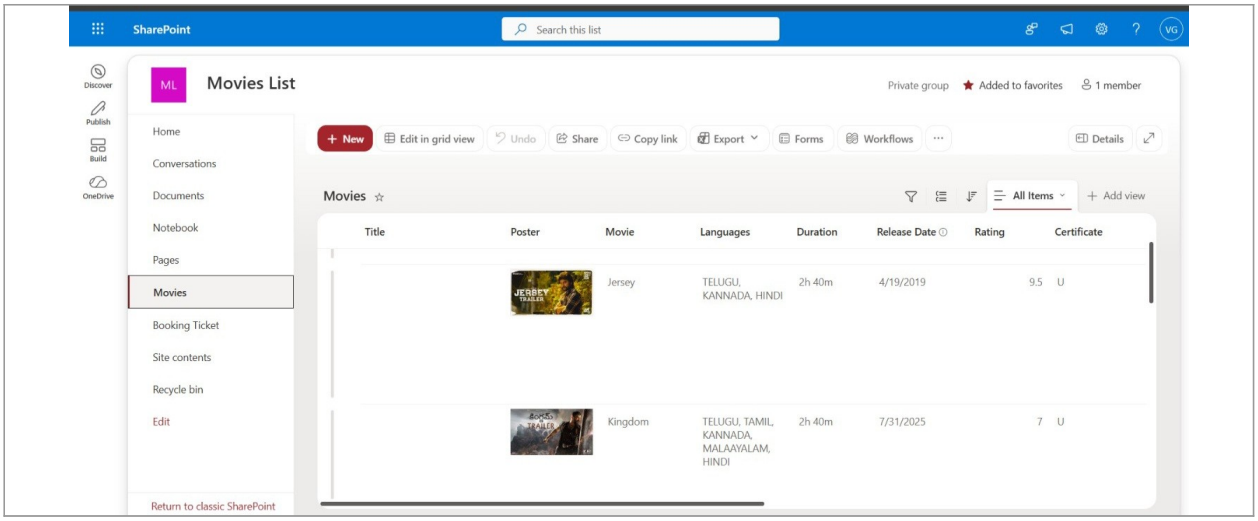
Text Input controls for Name, Email, Address; Labels showing selected offer's Title, Description, Discount, and Image; Submit Button running Patch() into the Offers Customer Details list.

Part B: SharePoint Backend Lists

All app data is stored in SharePoint Lists, which act as the database for the Power Apps front-end. Each list below corresponds to a specific feature of the app, and each one is referenced by at least one screen in Part A.

Screen 1 of 9

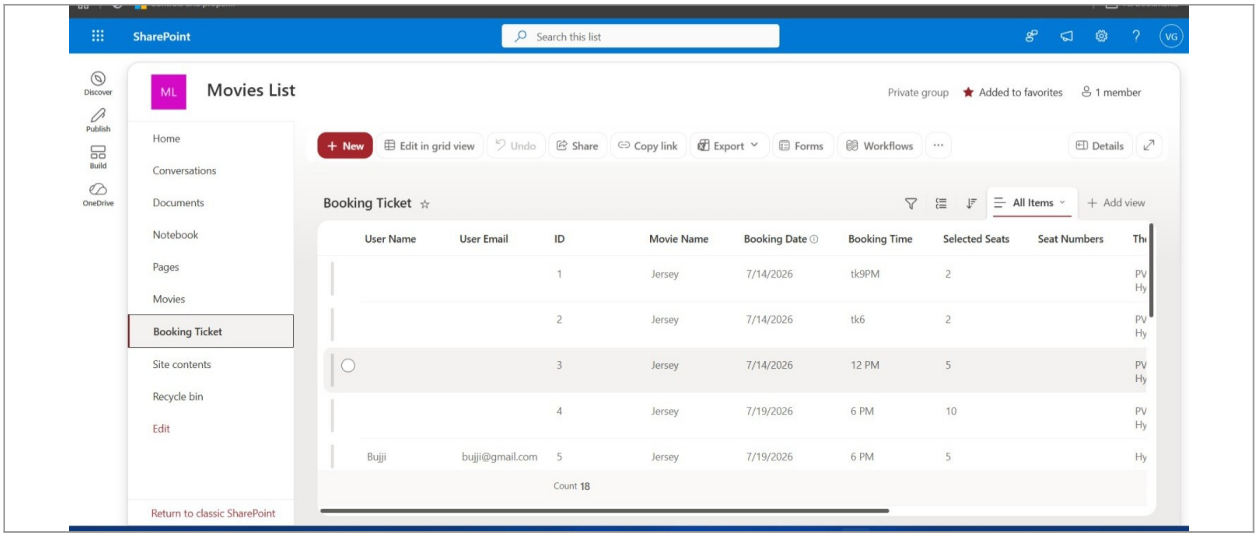
1. Movies List (SharePoint)



The Movies List is the master data source behind the Movies (Films) screen in the app. It stores one row per film, with columns for Title, Poster image, Movie name, Languages, Duration, Release Date, Rating, and age Certificate. The Poster column stores an image file that is displayed inside the Gallery control on the Movies screen and again on the Book Movie screen. The Languages column holds multiple values, for example TELUGU, KANNADA, HINDI, letting a single movie be tagged as available in more than one language. The Rating column is a numeric field, shown here as values like 9.5 or 7, which the app converts into a star display on the Book Movie screen. The Release Date column is used both for display purposes and potentially for sorting or filtering upcoming versus already-released movies. The Certificate column stores classification values such as 'U', which the app displays directly next to the movie's other details. Every time a new movie is added as a row in this list, it automatically appears in the app's Movies gallery without any changes to the Power Apps code, since the gallery is bound directly to this list. This is a good list to reference in an interview when explaining how SharePoint acts as a lightweight, no-code database for a Power Apps solution. It also shows how a single list can serve multiple screens (Movies list screen and the Book Movie detail screen) just by passing the selected record between them.

Screen 2 of 9

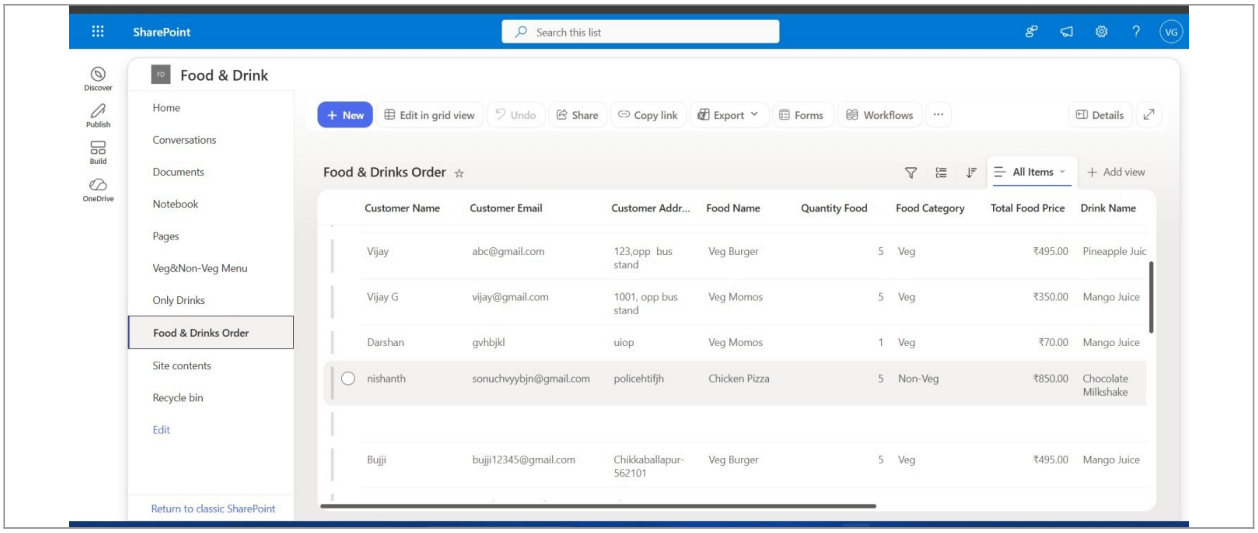
2. Booking Ticket List (SharePoint)



The Booking Ticket List records every seat booking a user completes through the app. Each row captures the User Name and User Email of the customer who booked, along with a unique ID for the booking. The Movie Name column stores which film was booked, linking this list conceptually back to the Movies List, though not necessarily through a formal SharePoint lookup relationship. The Booking Date and Booking Time columns record the exact show date and time slot the customer selected on the Seat Booking screen. The Selected Seats column stores how many seats were booked in that transaction, for example 2, 5, or 10 seats in a single booking. A Seat Numbers column (visible as a partially cut-off column in the list view) stores which specific seat numbers were chosen from the 40-seat grid. This list is written to from the Confirm Booking / Payment screen, once the user finalises their seat selection and confirms payment. Some rows show blank User Name and User Email fields, which suggests those bookings may have been created for testing or before a login/identification step was properly wired into the Patch formula. The Count shown at the bottom of the list view (18) tells us the total number of booking records currently stored, useful for tracking how many transactions the app has processed during testing. This list is one of the most important tables to highlight in an interview, since it represents the core transactional data of the entire application. It is also a good example for discussing data integrity, since ensuring every booking row has complete user, movie, and seat information is critical for a real booking system.

Screen 3 of 9

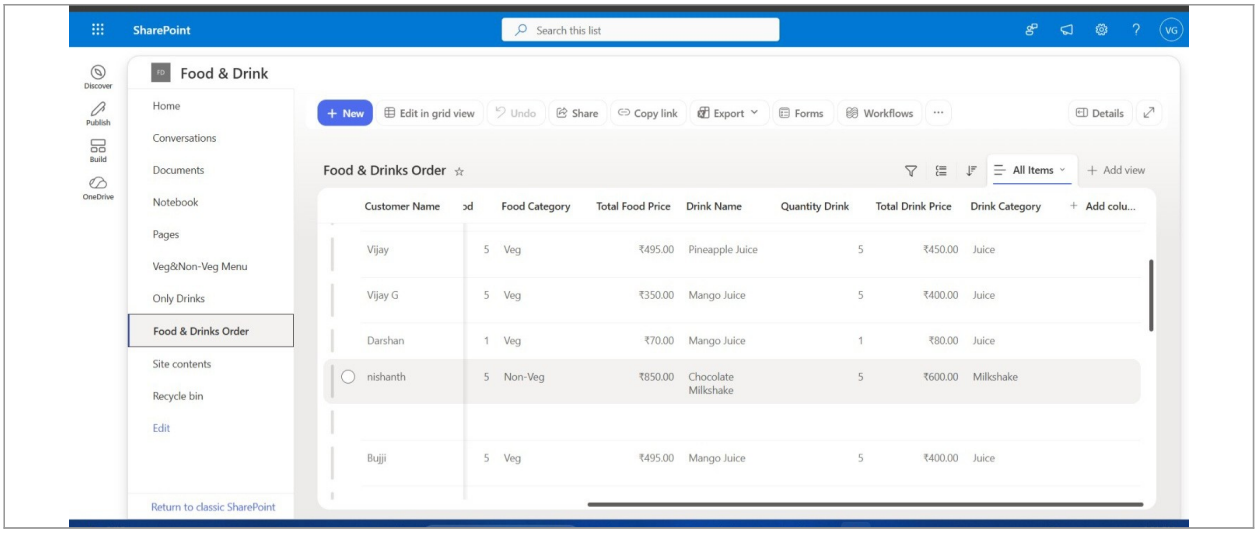
3. Food & Drinks Order List — Order Summary (SharePoint)



This is one view of the Food & Drinks Order list, focused on the order summary details for each item purchased. Each row represents a single item within an order, showing the Customer Name who placed it, such as Vijay, Darshan, or Bujji. The Food Category column classifies each item as Veg or Non-Veg, letting the business (or the app) easily filter dietary preferences. The Total Food Price column stores the calculated cost for the quantity of food ordered, for example ₹495.00 for 5 units of a Veg item. The Drink Name and Quantity Drink columns record which beverage was ordered alongside the food and how many units were requested. The Total Drink Price column stores the calculated cost of the drink portion of the order, separate from the food cost. The Drink Category column further classifies the beverage type, for example Juice or Milkshake, matching the categorisation used in the Only Drinks list. This list is populated from the Add to Cart screen when a user finalises their food and drink selections using the SHOP button. Because food and drink pricing are tracked in separate columns, the app (or a report built on top of this list) can calculate food revenue and drink revenue independently. This view of the list is useful in an interview for showing how a single list can support multiple views tailored to different reporting needs, without duplicating the underlying data.

Screen 4 of 9

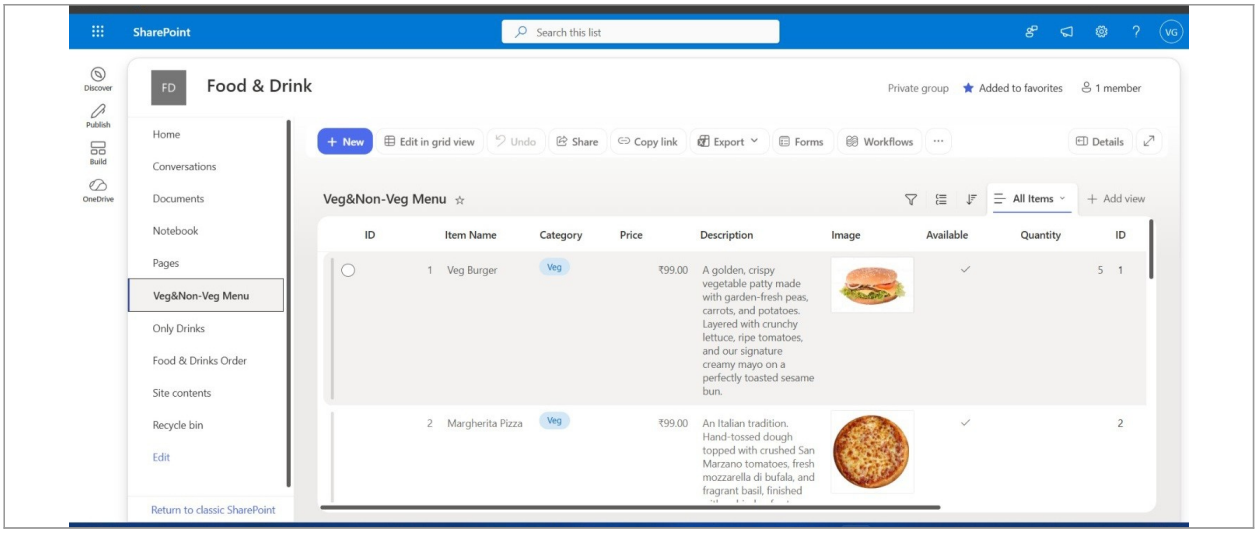
4. Food & Drinks Order List — Customer Details (SharePoint)



This is a second view of the same Food & Drinks Order list, this time focused on customer contact and delivery details. Alongside Customer Name, this view adds a Customer Email column, for example vijay@gmail.com or bujji12345@gmail.com, captured from the text input fields on the Add to Cart screen. A Customer Address column stores the delivery address entered by the user, for example '123, opp bus stand' or 'Chikkaballapur-562101'. The Food Name column in this view shows the specific item ordered by name, such as Veg Burger, Veg Momos, or Chicken Pizza, rather than only the category. Quantity Food, Food Category, and Total Food Price columns repeat here to give a complete picture of what was ordered and its cost, alongside the Drink Name for that same order. Having both a summary view (Part B, screen 3) and a customer-details view (this screen) of the same underlying list is a common SharePoint technique to avoid creating duplicate lists for the same data. This view would typically be used by whoever is responsible for preparing and delivering the food order, since it has all the contact information needed to fulfil the order. Some rows show placeholder-looking data, such as 'uiop' as an address, which suggests this was tested during development rather than reflecting real customer submissions. This list demonstrates a practical real-world skill: designing one list that can serve two different audiences (operations staff needing contact details, and finance/reporting needing price summaries) through separate views. This is a strong point to raise in an interview when asked about SharePoint list design best practices, since it shows list views being used efficiently instead of over-engineering the data model.

Screen 5 of 9

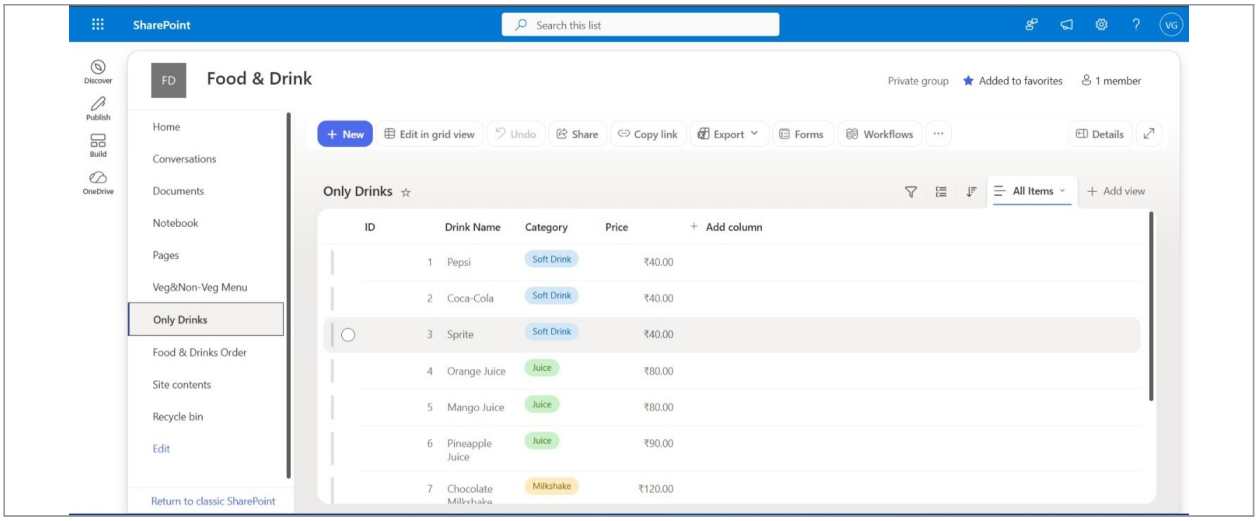
5. Veg & Non-Veg Menu List (SharePoint)



The Veg & Non-Veg Menu List is the master food menu that powers the food-side Gallery on the Food & Drink screen. Each row has an ID, an Item Name such as Veg Burger or Margherita Pizza, and a Category column tagging the item as Veg (shown with a green badge in the list view). The Price column stores the cost of each item, for example ₹99.00 for both the Veg Burger and Margherita Pizza shown here. A Description column holds a longer marketing-style write-up for each item, for example describing the Veg Burger as a 'golden, crispy vegetable patty... layered with crunchy lettuce, ripe tomatoes'. An Image column stores the actual photo of the dish, which is what gets displayed next to each item inside the app's Food & Drink gallery. An Available column, shown as a checkmark, lets the business mark whether an item is currently in stock or temporarily unavailable without deleting the row. A Quantity column tracks how many units of that item are currently available or have been ordered, depending on how the app's logic uses this field. A second ID column appears further to the right, which may be used as an internal reference key separate from the list's default ID, useful for linking with the order lists. Because the gallery on the Food & Drink screen reads directly from this list, adding a new dish here (with its price, description, and image) makes it instantly available in the app. This list is a good example in an interview of using a SharePoint list not just for storing simple text and numbers, but also rich content like descriptions and images to drive an app's UI.

Screen 6 of 9

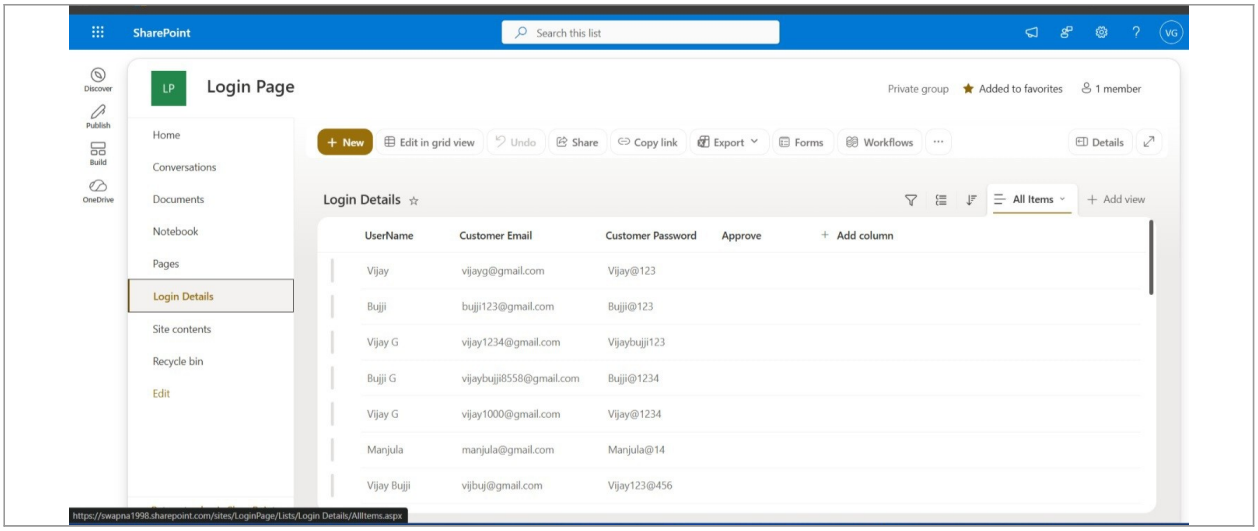
6. Only Drinks List (SharePoint)



The Only Drinks List is a dedicated master list of all beverages available for order, separate from the food menu list. Each row has an ID, a Drink Name such as Pepsi, Coca-Cola, Sprite, Orange Juice, Mango Juice, Pineapple Juice, or Chocolate Milkshake. The Category column classifies each drink as Soft Drink, Juice, or Milkshake, using coloured badges to make the categories easy to scan visually in the list view. The Price column stores the cost of each drink, ranging from ₹40.00 for soft drinks up to ₹120.00 for a Chocolate Milkshake in this example. This list feeds the drinks-side Gallery on the Food & Drink screen, meaning the app automatically reflects any new drink or price change made here. Keeping drinks in their own list, rather than merging them into the Veg & Non-Veg Menu list, keeps the data model simpler since drinks don't need Veg/Non-Veg or Available/Description columns in the same way food items do. This separation also makes it easier to build a dedicated 'Only Drinks' view or report if the business wants to analyse beverage sales independently of food sales. This list is small and straightforward compared to the food menu list, which makes it a good simple example to open with in an interview before moving into the more complex lists. It reinforces the idea that not every piece of app data needs a complicated schema; sometimes a simple three or four column list is the right design choice. The category-based colour coding used here is also implemented purely through SharePoint's own choice-column formatting, without needing any extra Power Apps logic.

Screen 7 of 9

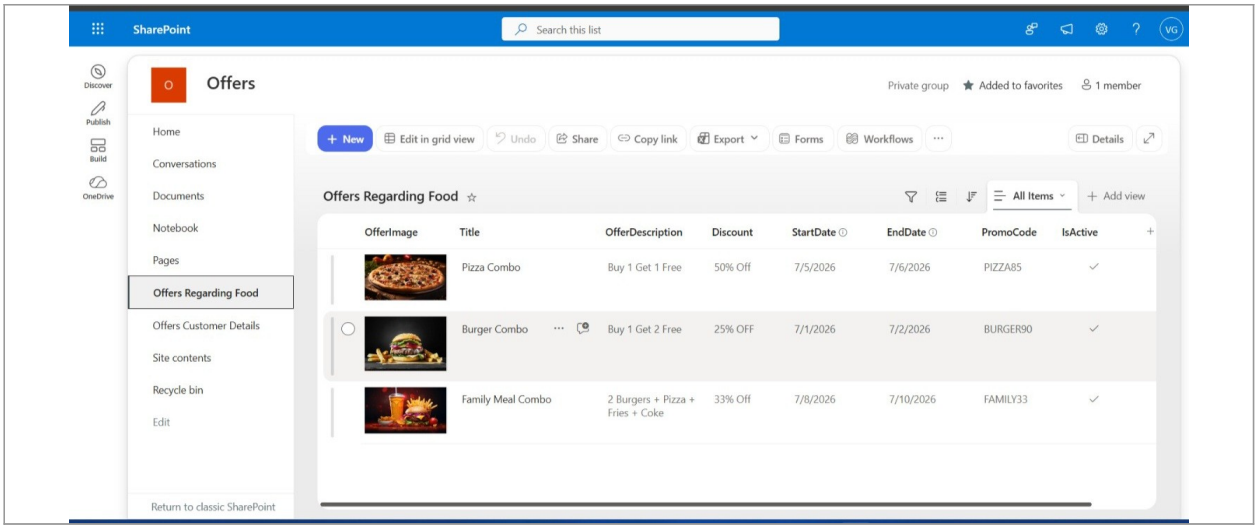
7. Login Details List (SharePoint)



The Login Details List stores every registered user's credentials and is the backbone of the app's authentication system. Each row has a UserName column, for example Vijay, Bujji, or Manjula, matching what the user entered during registration. A Customer Email column stores each user's email address, which is used as the primary identifier when logging in. A Customer Password column stores the password the user set, shown here in plain text values such as 'Vijay@123' or 'Manjula@14'. An Approve column exists but appears empty in most rows here, suggesting this could be intended for an admin approval workflow that may not yet be actively used. New rows are added to this list whenever a user completes the New Register screen, and existing rows are read from during Login and updated during Reset Password. Since passwords are stored here as plain text rather than hashed, this is an important point to raise in an interview: in a real production system, storing passwords in plain text inside a SharePoint list would be a serious security risk, and a proper solution would use Azure AD authentication or at least some form of encryption or hashing. Being able to identify this kind of limitation, and explain how it should be solved in production, shows the interviewer a mature understanding of app security rather than just app-building mechanics. This list is also useful to describe how Filter or LookUp formulas at the Login Page are used to match entered Email and Password values against this list's rows. Overall, this list demonstrates the simplest possible way to implement authentication in a Power Apps prototype, which is appropriate for a training or demo project but would need to be replaced by a more secure method for real deployment.

Screen 8 of 9

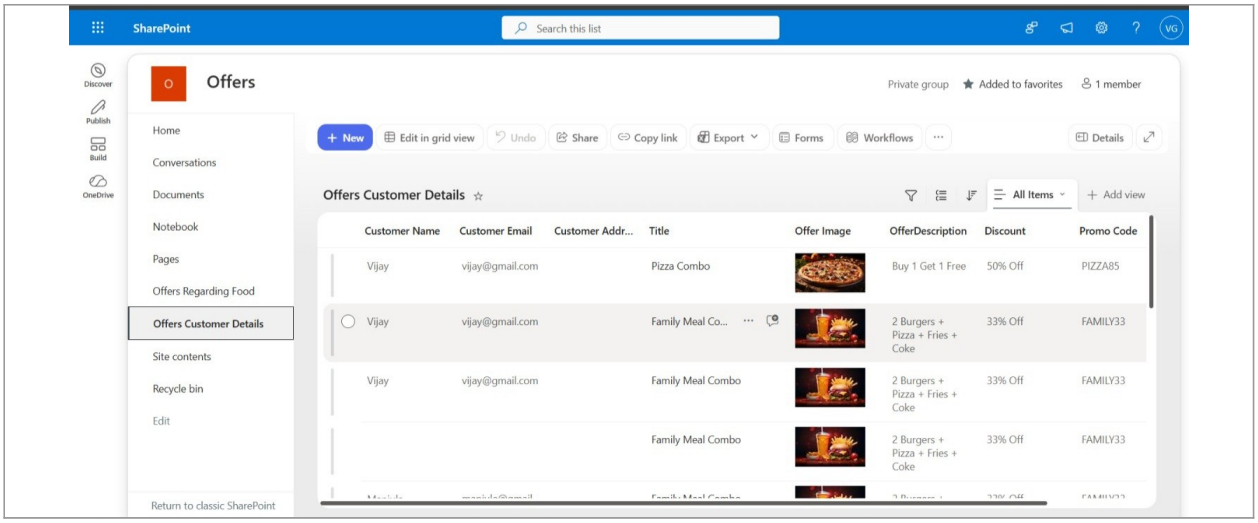
8. Offers Regarding Food List (SharePoint)



The Offers Regarding Food List is the master list of all promotional deals shown on the app's Offers screen. Each row includes an OfferImage, a Title such as Pizza Combo, Burger Combo, or Family Meal Combo, and an OfferDescription summarising what the deal includes. The Discount column stores the percentage off, for example '50% Off' or '25% OFF', displayed prominently in the app's Offers gallery. StartDate and EndDate columns define the validity window for each offer, for example the Pizza Combo running from 7/5/2026 to 7/6/2026. A PromoCode column stores the code customers can use to redeem the discount, such as PIZZA85 or BURGER90. An IsActive column, shown with a checkmark, lets the business toggle whether an offer is currently live without deleting the historical row. The Offers screen's Gallery control is expected to filter this list so that only rows where IsActive is true (and optionally where the current date falls between StartDate and EndDate) are shown to users. This design lets the business plan and pre-load future offers into the list ahead of time, and they will only appear to app users once their start date arrives and IsActive is switched on. This is a good list to discuss in an interview when explaining time-based or condition-based filtering logic in Power Apps, since it goes beyond simple static displays. It also shows good separation of concerns: this list defines what offers exist, while the next list (Offers Customer Details) tracks who actually redeemed them.

Screen 9 of 9

9. Offers Customer Details List (SharePoint)



The Offers Customer Details List logs every offer redemption submitted through the app's Offer Cart screen. Each row records the Customer Name and Customer Email of whoever redeemed an offer, for example Vijay at vijay@gmail.com. A Customer Address column captures the delivery or contact address entered at the time of redemption. The Title column repeats the name of the offer that was redeemed, such as Pizza Combo or Family Meal Combo, matching the Title in the Offers Regarding Food list. An Offer Image column stores a copy of the offer's image for reference, so this list does not need to look the image up from the other list every time it is viewed. The OfferDescription and Discount columns similarly duplicate details from the original offer, for example 'Buy 1 Get 1 Free' with '50% Off', or '2 Burgers + Pizza + Fries + Coke' with '33% Off'. A Promo Code column stores which code was actually used at the time of redemption, letting the business verify that the correct discount was applied. Storing a copy of the offer's details here, rather than only storing a reference back to the Offers Regarding Food list, means this history remains accurate even if the original offer is later changed or removed. This is a useful list to bring up in an interview when discussing denormalisation, a common real-world data design decision where some duplication is accepted in exchange for more reliable historical records. Together with the Booking Ticket and Food & Drinks Order lists, this list completes the transactional side of the app's SharePoint backend, capturing every meaningful customer action alongside the master data lists that define movies, menu items, and offers.